

Chap.8 오토인코더 (Auto-encoder)

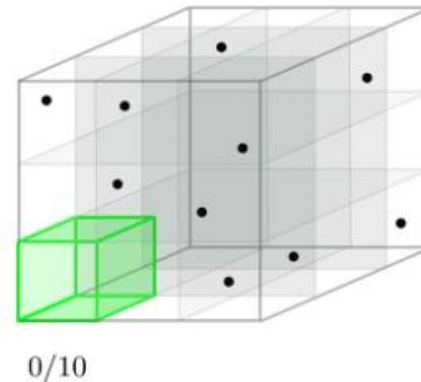
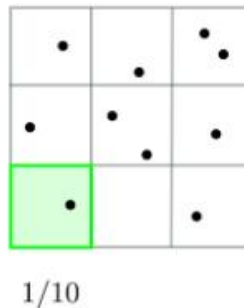
방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

■ 차원의 저주 (The Curse of Dimensionality)

- 저차원에서는 발생하지 않고, 고차원 공간에서 데이터를 분석할 때 발생하는 문제들
- 데이터 밀도가 희소 **=> 머신러닝 적용 시 문제점?**
- 저장 공간이 비효율적

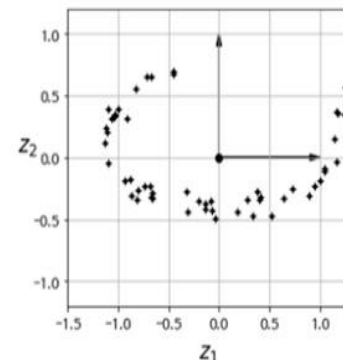
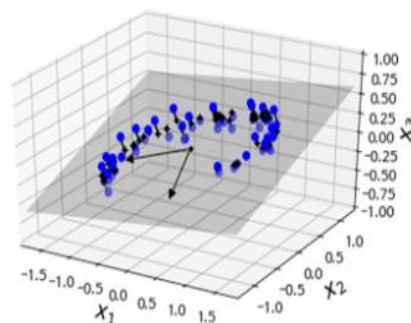


대표적인 차원 축소 기법



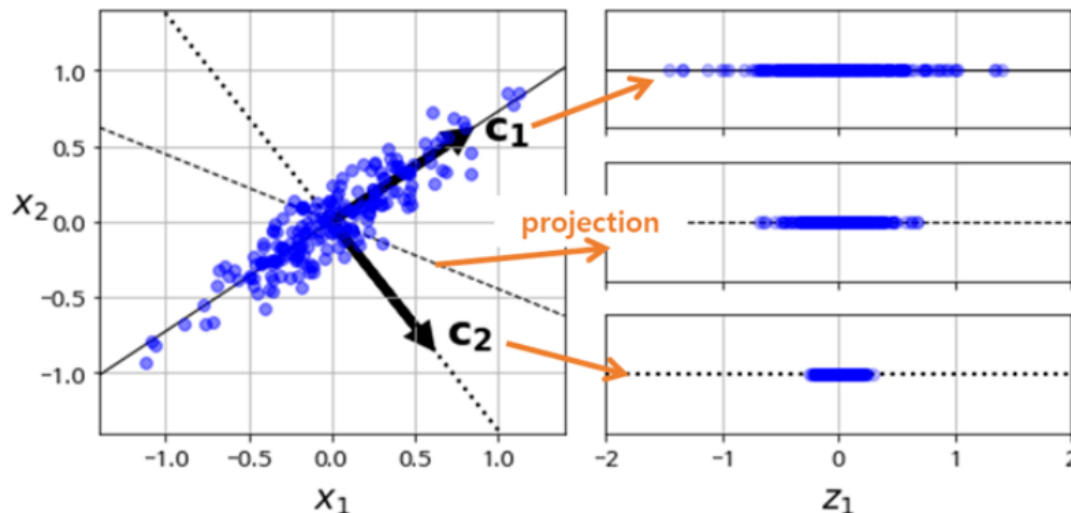
■ 투영 (Projection)

- 고차원에서 하나의 차원축을 삭제

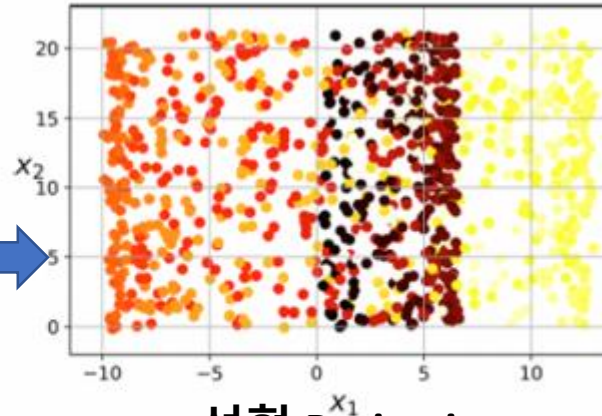
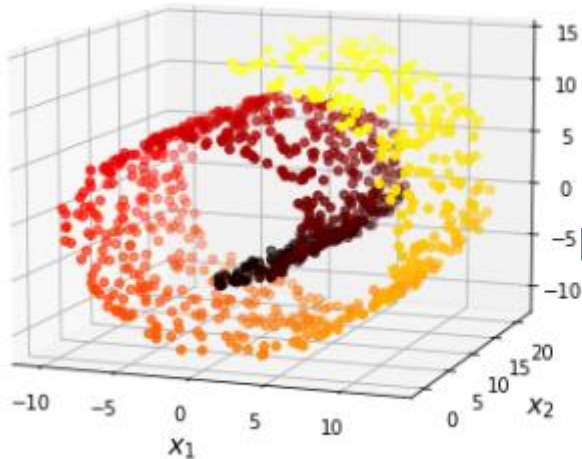


■ 주성분 분석 (Principal Component Analysis, PCA)

- 데이터에 가장 가까운 초평면(Hyperplane)을 구한 다음, 해당 초평면에 투영

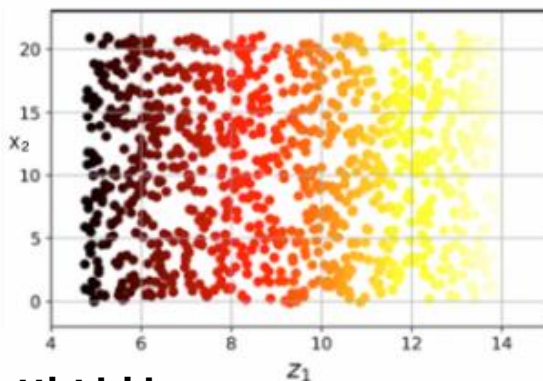


[참고] Manifold



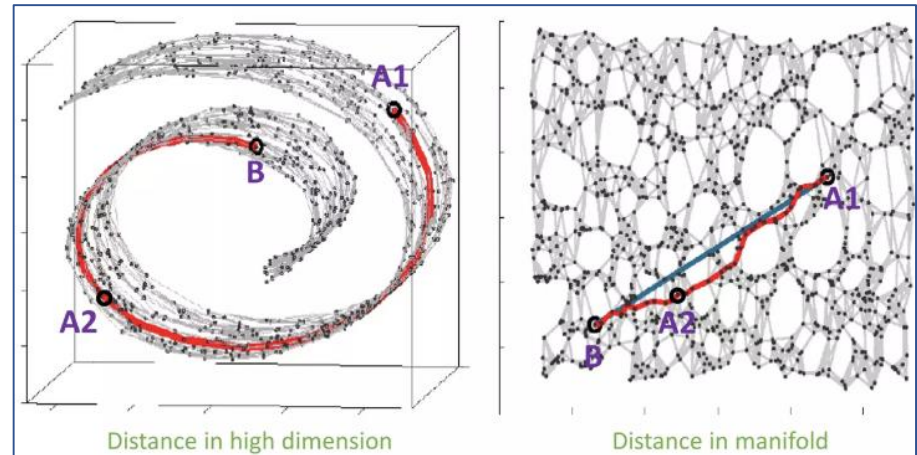
선형 Projection

Swiss roll data



비선형 Projection

=> Manifold: 고차원 공간에서 접거나 비틀어서 생성하는 저차원 공간



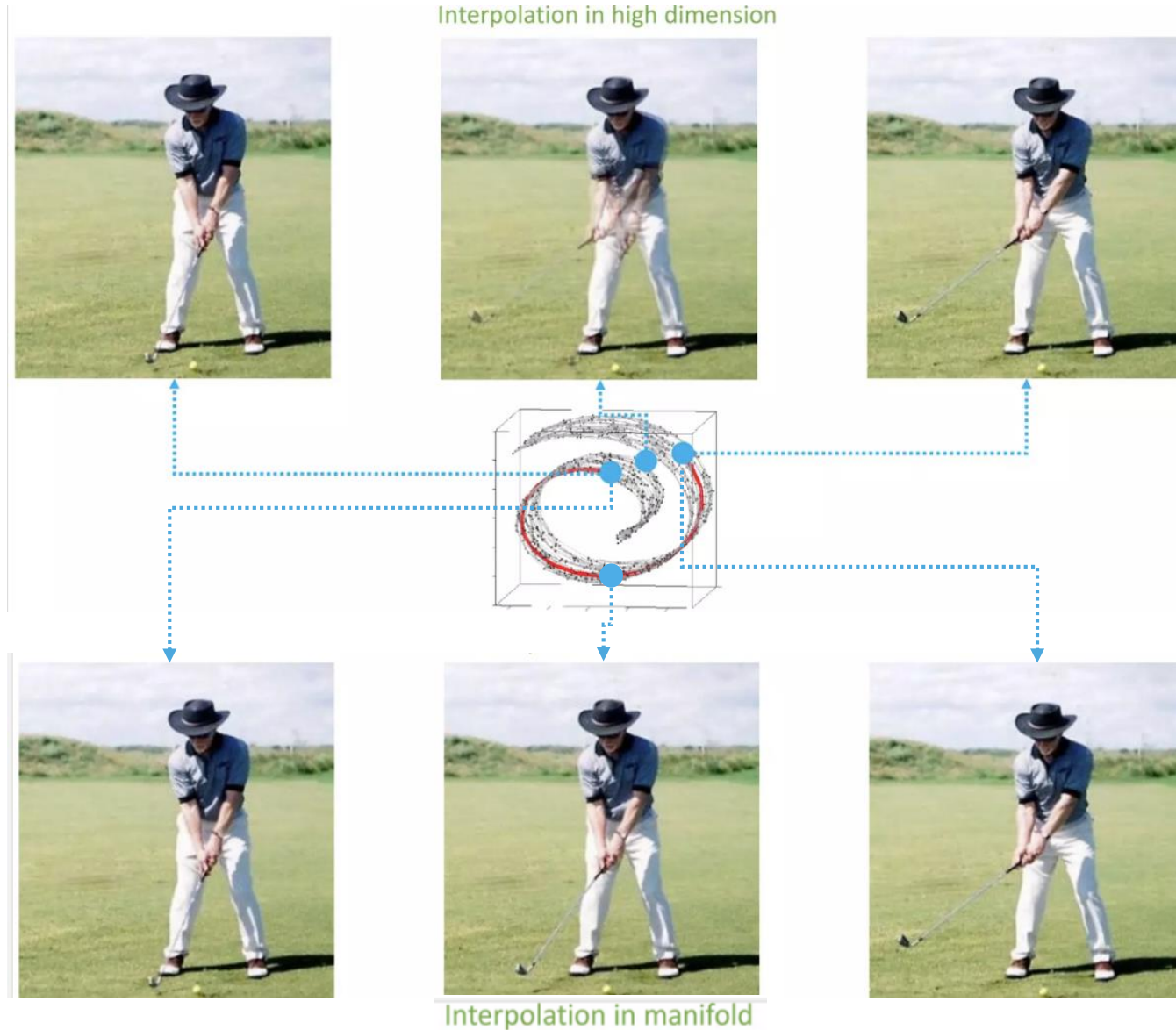
[Manifold Learning]

=> 데이터의 경향성을 부각하는 Manifold를 찾는 방법

[참고] Manifold Learning



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.



Encoder & Decoder

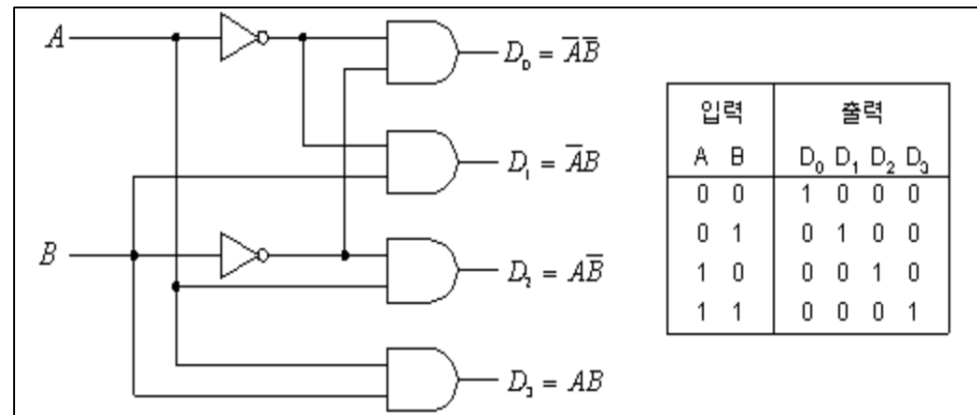
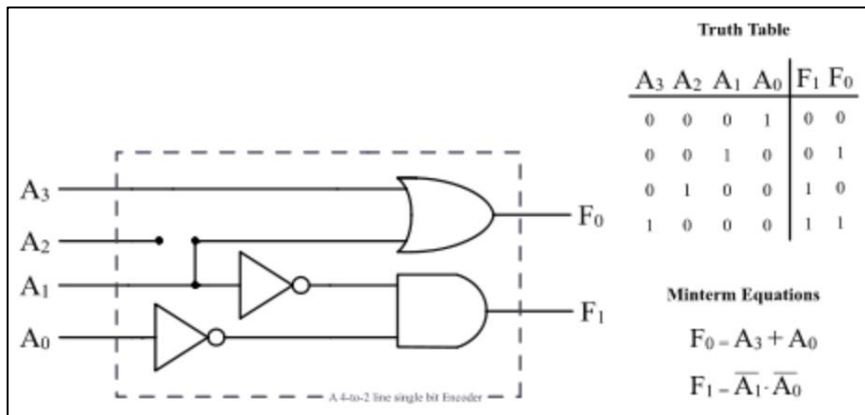


Encoder

- 2^n 비트의 정보를 n 개의 출력으로 "암호화"하는 논리회로

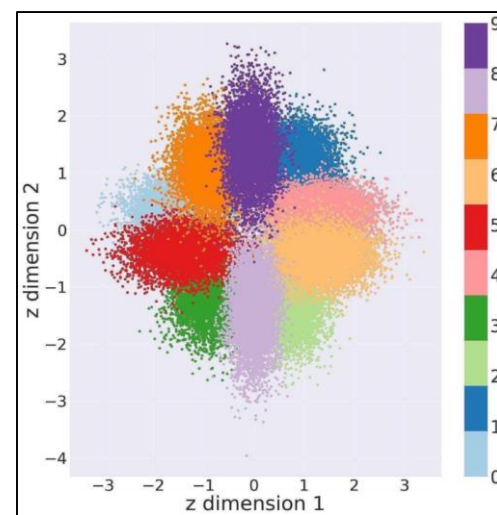
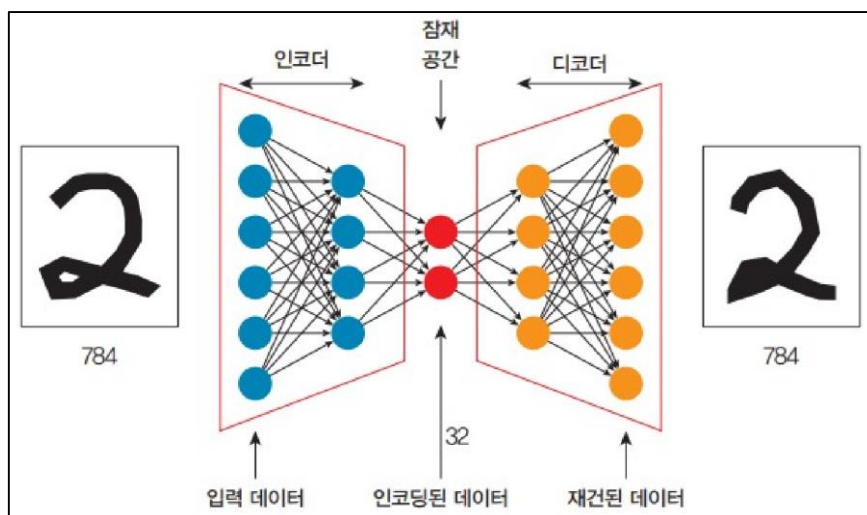
Decoder

- n 개의 입력 정보를 2^n 개의 출력으로 "해독"하는 논리회로



■ 오토인코더 (Auto-Encoder, AE)

- 목적: 고차원 데이터를 저차원 벡터로 축소하는 Encoder와 축소된 벡터를 원래 데이터로 복원하는 Decoder를 신경망으로 자동 설계
- Encoder: Recognition Network라고도 부르며, 차원 축소 기능
- Decoder: Generative Network라고도 부르며, 차원 복원 기능
- Latent Space: 차원이 축소된 벡터들의 공간

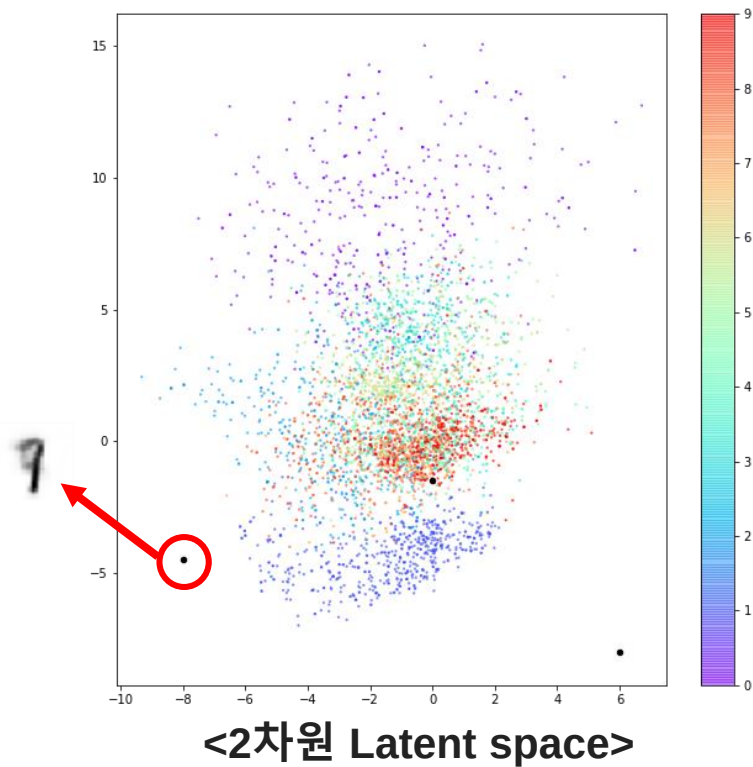


Latent space

Latent space



Latent Space 예시

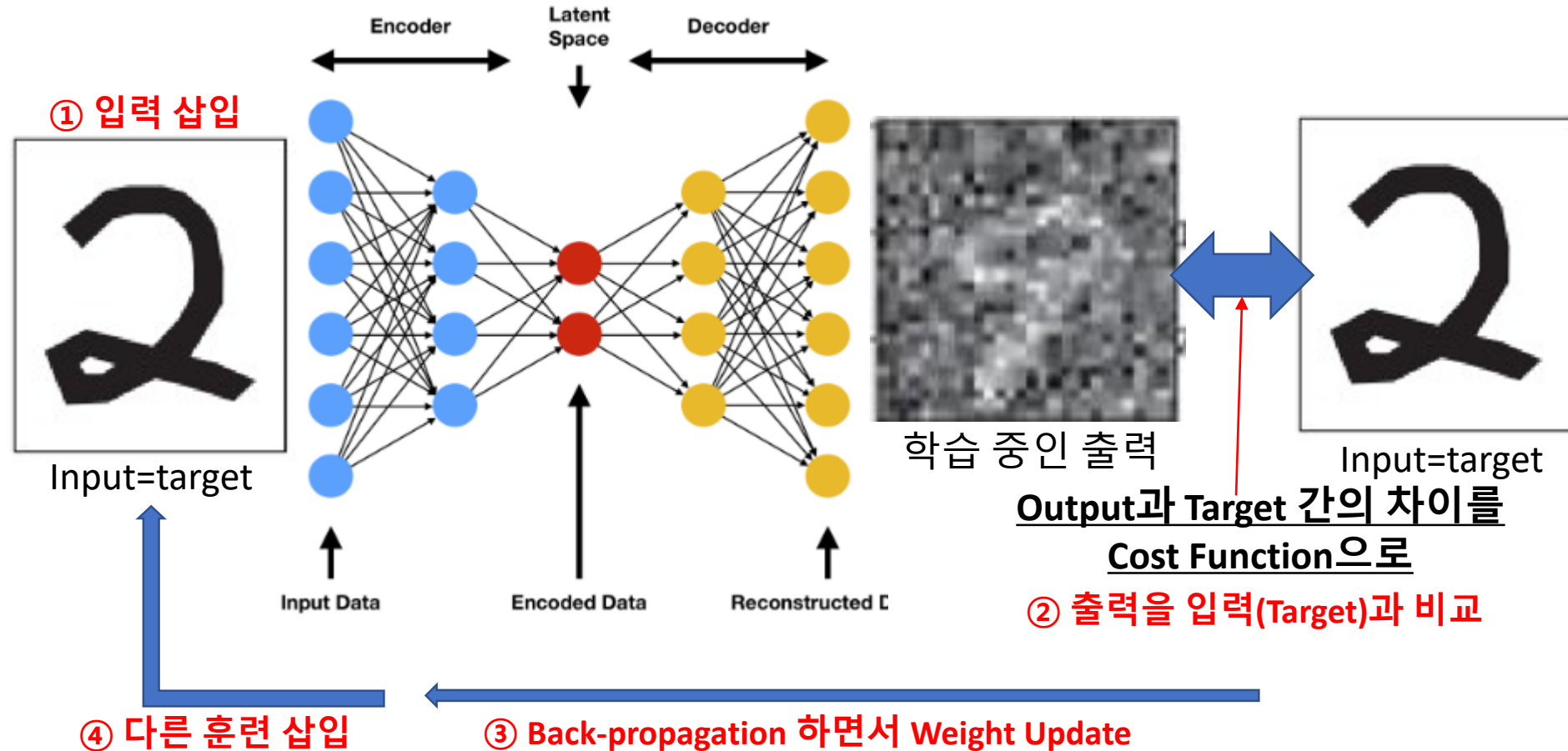


Auto-Encoder 학습 원리



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ 학습 원리



필기체 숫자 압축

- MNIST 데이터셋 취득

```
mnist = keras.datasets.mnist
(x_train, _), (x_test, _) = mnist.load_data()
#label data 필요 없음
x_train = x_train/255
x_test = x_test/255
x_train = x_train.reshape((len(x_train), 784))
x_test = x_test.reshape((len(x_test), 784))
```

Dense만을 활용하기 위해 (1x784)로 변경
1차원 CNN 활용 가능

- Auto Encoder 모델 설계

입력을 28 by 28로 했을 경우 2차원 CNN 활용 가능

```
#AE 모델 설계
encoding_dim = 2 #잠재 벡터 차원 수
encoder = keras.models.Sequential([
    keras.layers.Dense(encoding_dim, activation='relu', input_shape = (784,))
])

decoder = keras.models.Sequential([
    keras.layers.Dense(784, activation='relu', input_shape = (2,))
])

AE = keras.models.Sequential([encoder,decoder])
```

Auto-Encoder 실습



- 필기체 숫자 압축
 - Auto Encoder 학습 및 예측

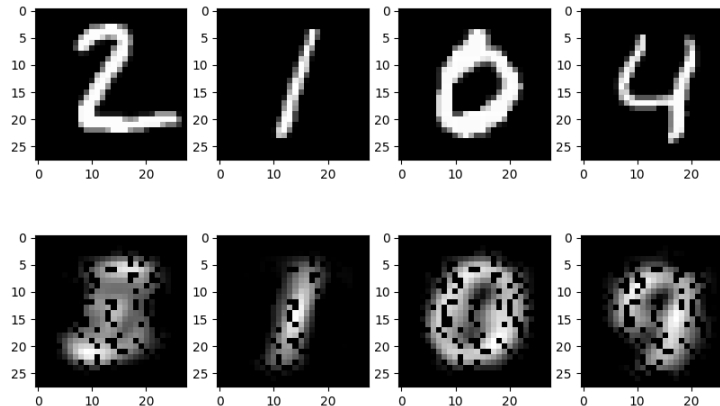
```
#학습
history = AE.fit(x_train,x_train, epochs = 50, batch_size = 256, shuffle=True)

#예측
decoded_img = AE.predict(x_test)

#plot
n = 4
plt.figure(figsize=(10,6))
for i in range(1, n+1):
    ax = plt.subplot(2,n,i)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray') #원본

    ax = plt.subplot(2, n, i+n)

    plt.imshow(decoded_img[i].reshape(28,28), cmap='gray')
plt.show()
```



▪ Functional API

- 기존에 사용하던 API는 Sequential API
- Sequential API는 layer를 차례대로 쌓아서 구성하는 방식
- Sequential API는 직관적이고 편리하지만 복잡한 모델 구현 불가
- Functional API는 각 층을 일종의 함수로 정의
- Functional API를 사용하여 Sequential API보다 유연한 모델 설계

```
inputs = keras.Input(shape=(10,))
hidden1 = keras.Dense(64, activation='relu')(inputs)
hidden2 = keras.Dense(64, activation='relu')(hidden1)
output = keras.Dense(1, activation='sigmoid')(hidden2)
model = keras.Model(inputs=inputs, outputs=output)
```

Functional API를 사용한 ANN

Chap.8 실습 (1)



각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #1) 앞선 실습에서 진행한 AE의 파라미터에 따른 결과 비교 분석

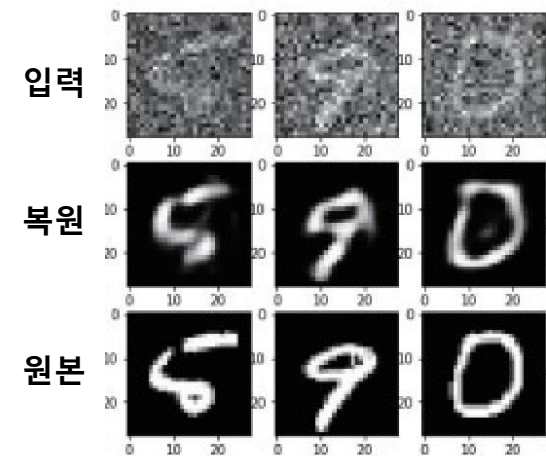
- Functional API를 사용하여 설계
- MNIST 데이터에 대해 AE 학습 진행 → AE의 입력 크기를 (28x28)로 할 것
- 잠재 벡터의 차원 수에 따른 결과 비교 분석
- Encoder 및 Decoder의 계층 추가 및 구조 변경에 따른 결과 비교 분석
- 다양한 방법으로 계층 변경
ex) Convolutional layer 추가 등등

실습 #2) 노이즈를 제거하는 AE 만들기

- Functional API를 사용하여 설계
- MNIST 데이터에 노이즈 추가
- 노이즈 데이터를 입력, 원본 데이터를 Target으로 진행
- 입력 이미지, 원본 이미지, 복원 이미지를 비교

```
noise_factor = 0.9
original_train = x_train
noise_train = np.random.normal(0,1, original_train.shape)
noisy_train = original_train + noise_factor * noise_train
```

노이즈 추가 예시



결과 예시

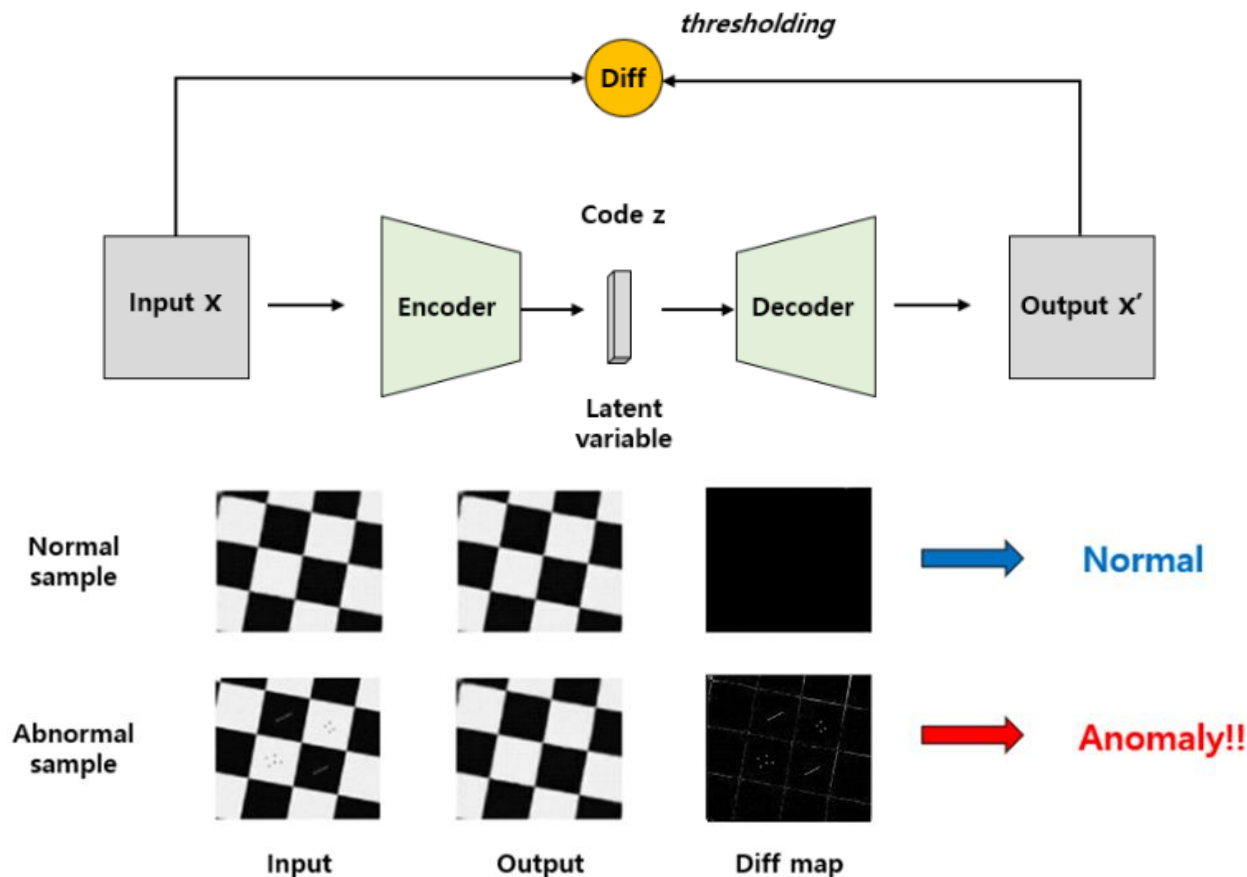
[AE의 활용] Anomaly Detection



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

▪ Autoencoder (AE) 기반 Unsupervised Anomaly Detection

- Input Image와 Reconstruction Image의 차이를 기반으로 이상치 탐지



Learning Memory-guided Normality for Anomaly Detection

CVPR 2020

Hyunjong Park*, Jongyoun Noh*, Bumsub Ham (*equal contribution)
Yonsei University

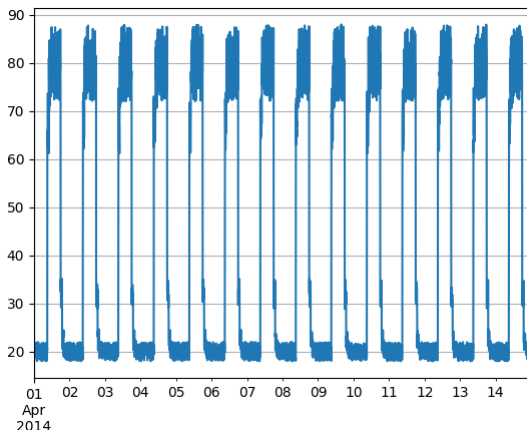
Chap.8 실습 (2)



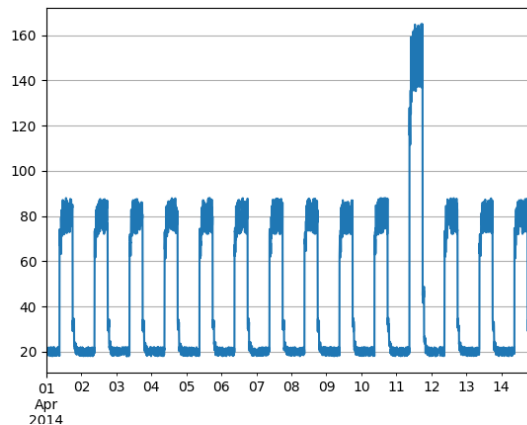
각 실습문제에 대해 *code(+ 주석), 그래프, 분석 내용* 등은 필수적으로 포함시킬 것

실습 #3) AE를 활용하여 Anomaly Detection을 구현

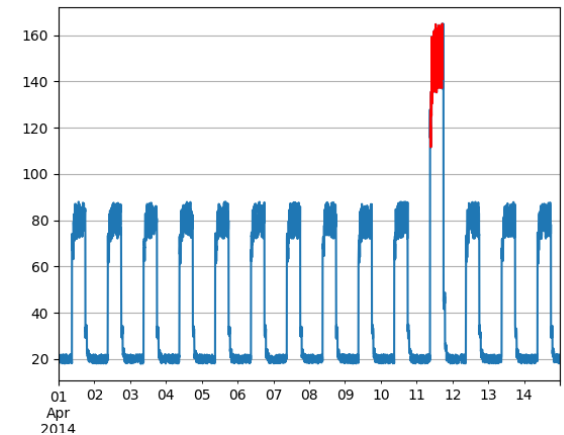
- 인공적으로 만들어진 시계열 데이터 활용
- 아래 Keras Link를 참고하여 시계열 데이터의 Anomaly Detection 구현
- 설명을 잘 숙지하고 여러 구조 및 파라미터 조절에 따른 결과 비교 분석 ex)threshold 등
- **단순 과제 제출 목적이 아닌, Anomaly Detection에 대한 이해를 목적으로 진행 요망**



정상 데이터



비정상 데이터



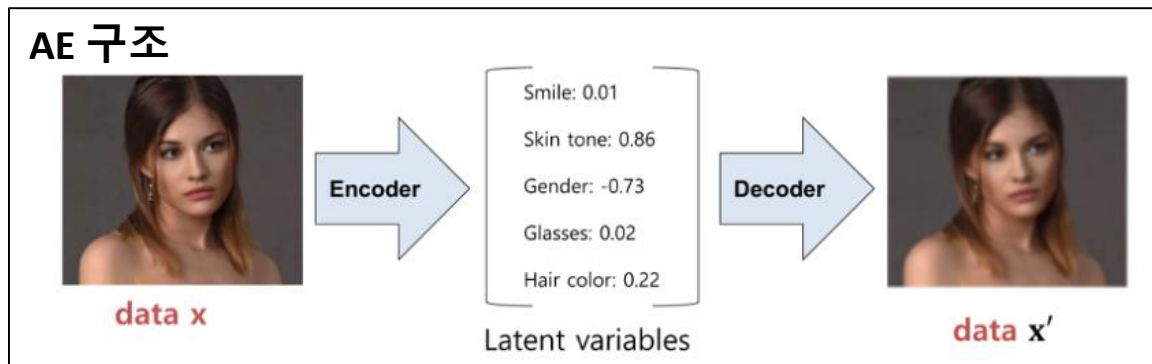
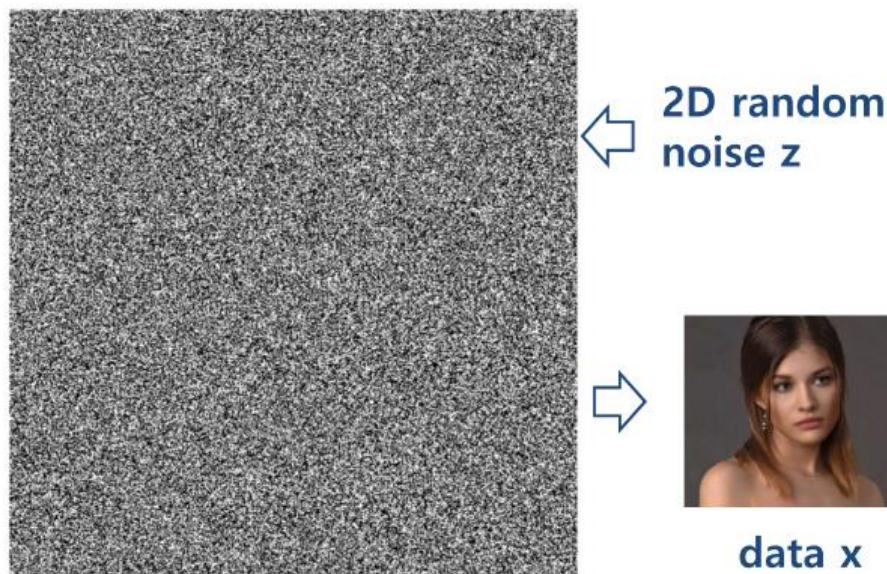
비정상 예측

https://keras.io/examples/timeseries/timeseries_anomaly_detection/

■ Motivation

VAE는 생성 모델의 근본

- 랜덤 노이즈로부터 새로운 이미지를 얻을 수 있을까?

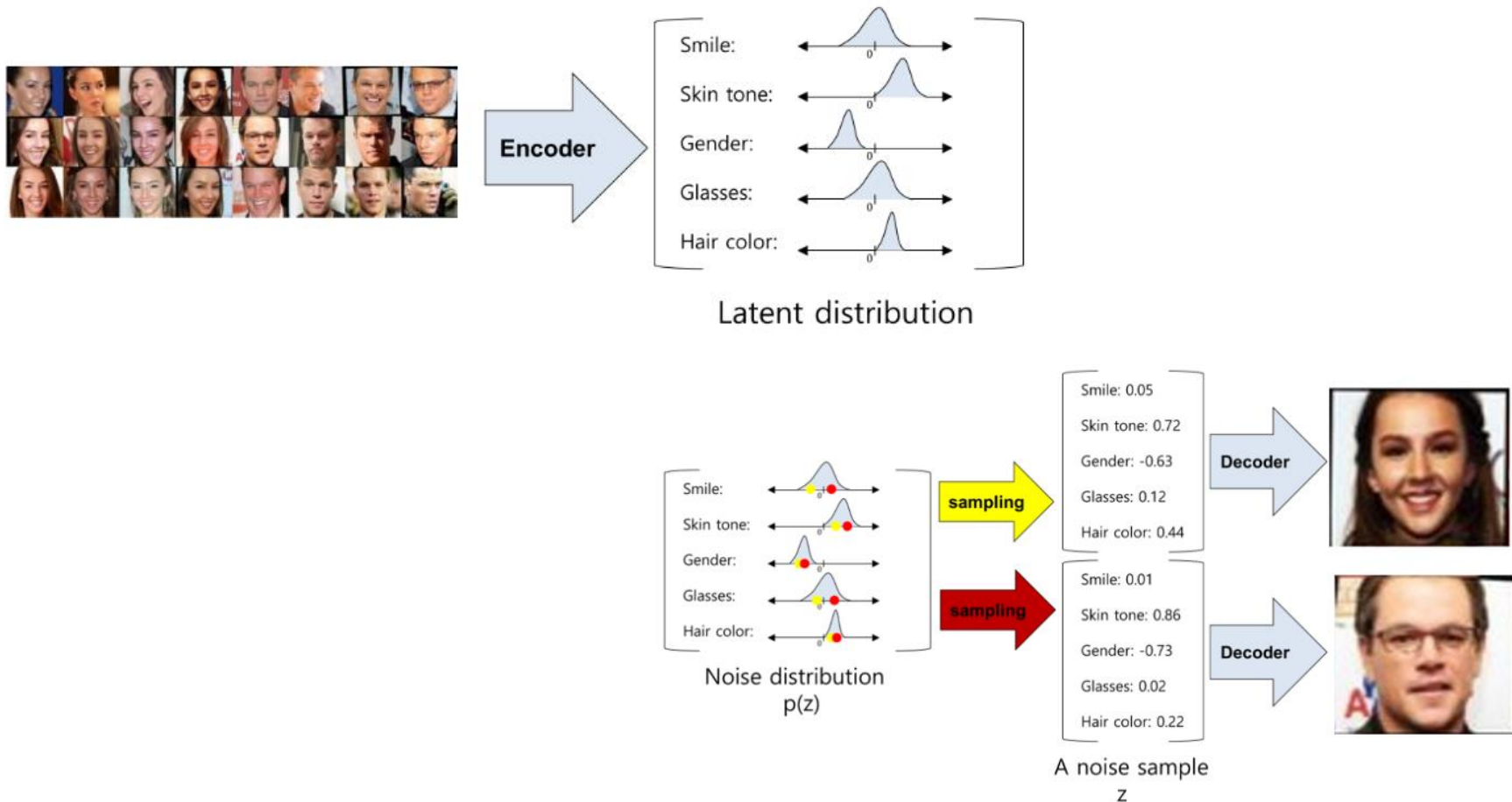


Variational Auto-Encoder



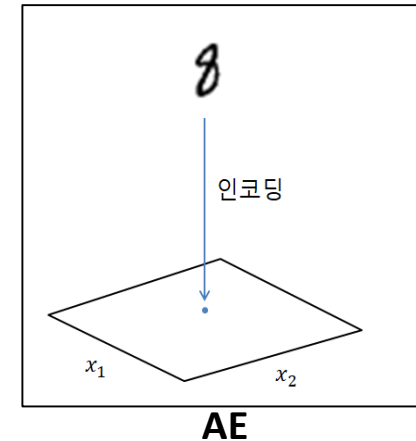
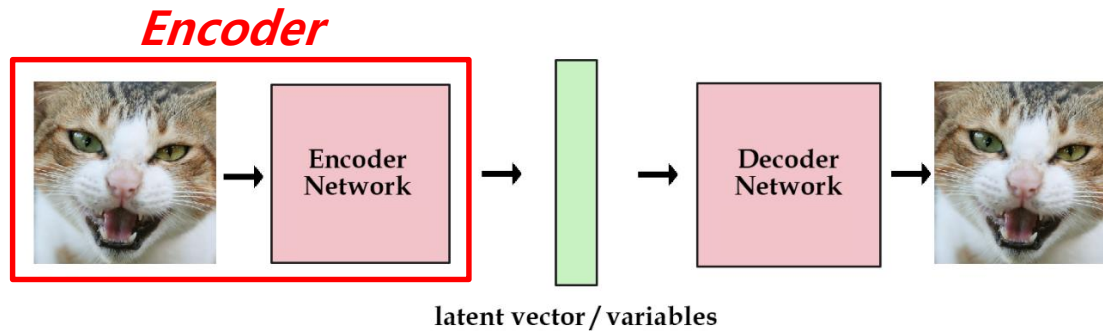
VAE의 구조

- Latent Space의 값을 확률 분포의 파라미터로 해석한다면?

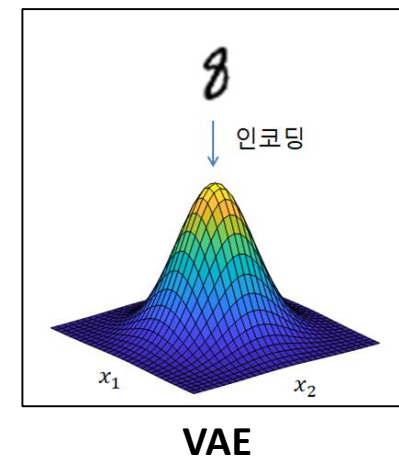
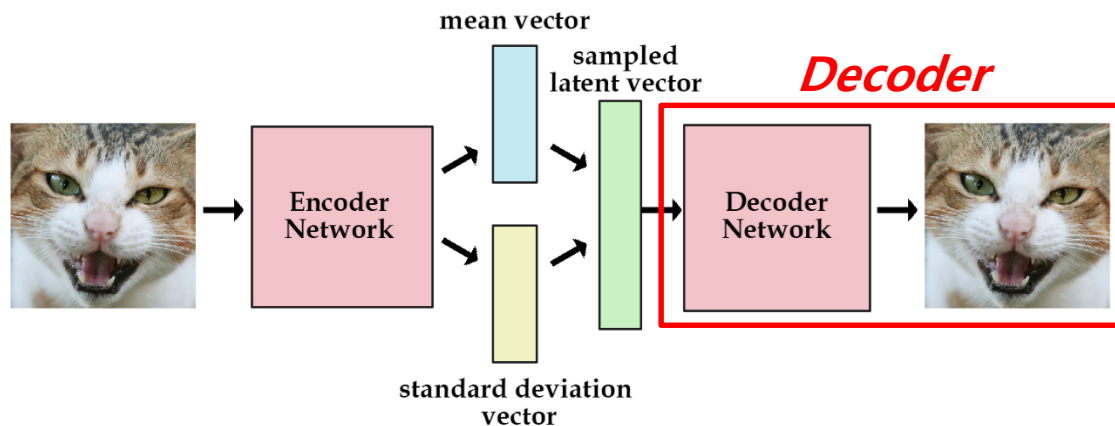


▪ AE(Auto Encoder) vs VAE(Variational Auto Encoder)

- AE의 목적 : Encoder를 통한 차원 축소



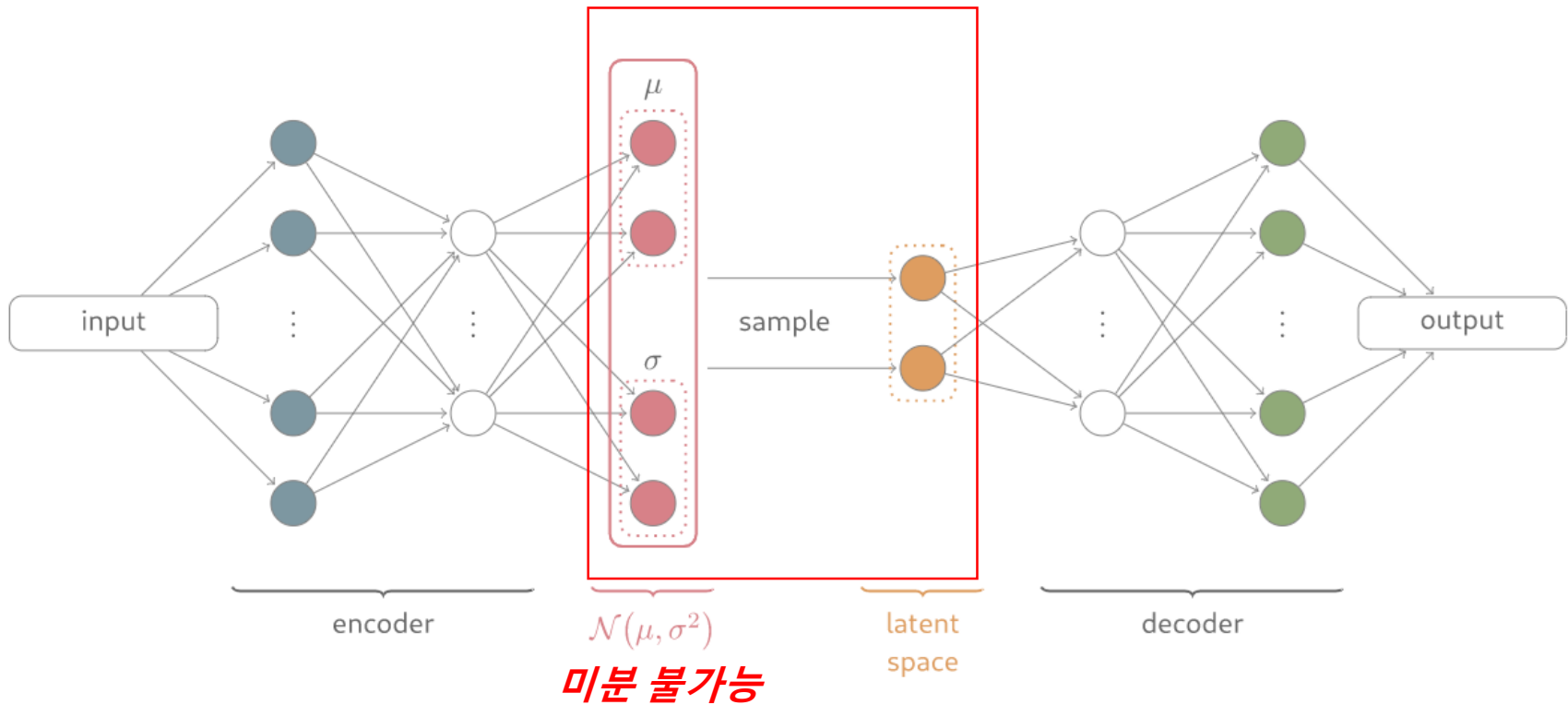
- VAE의 목적 : Decoder를 통한 새로운 데이터를 생성



Variational Auto-Encoder



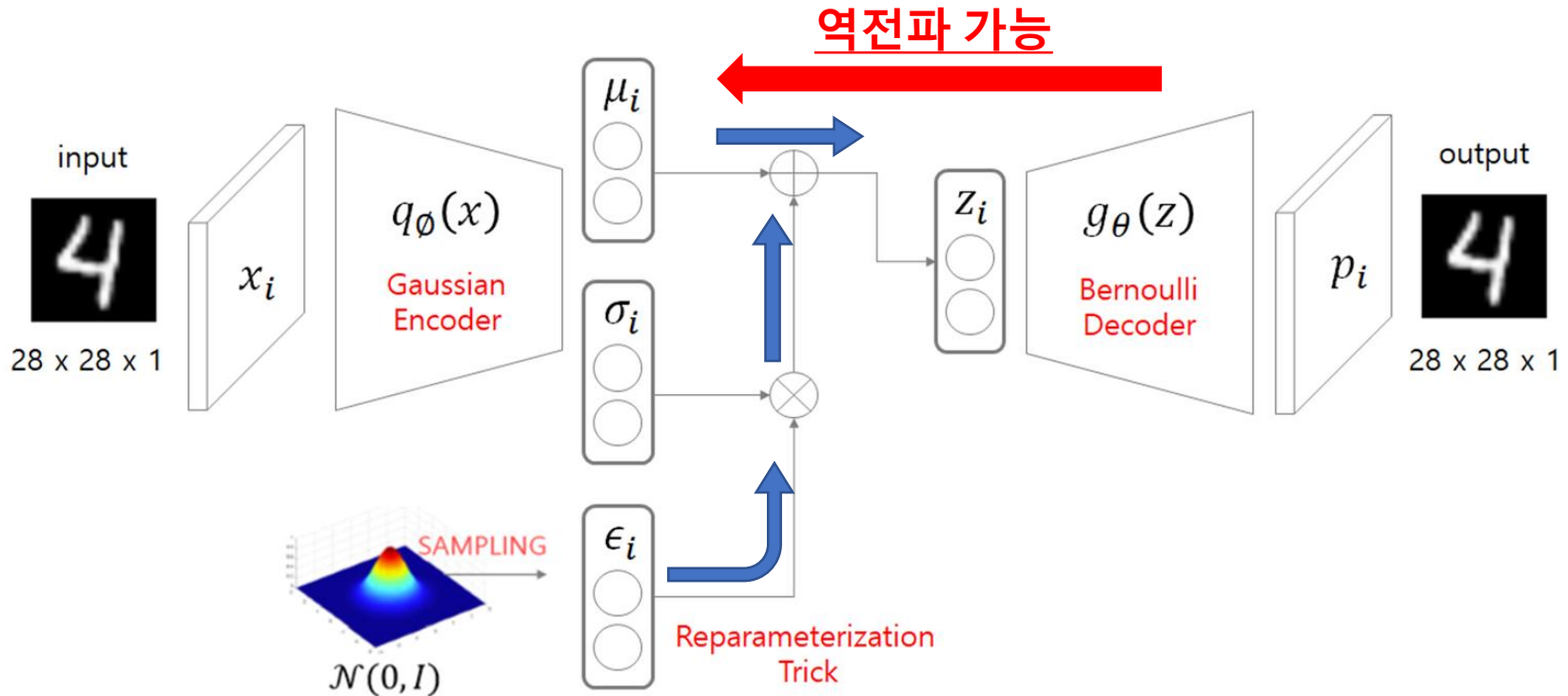
- VAE의 컨셉



Variational Auto-Encoder



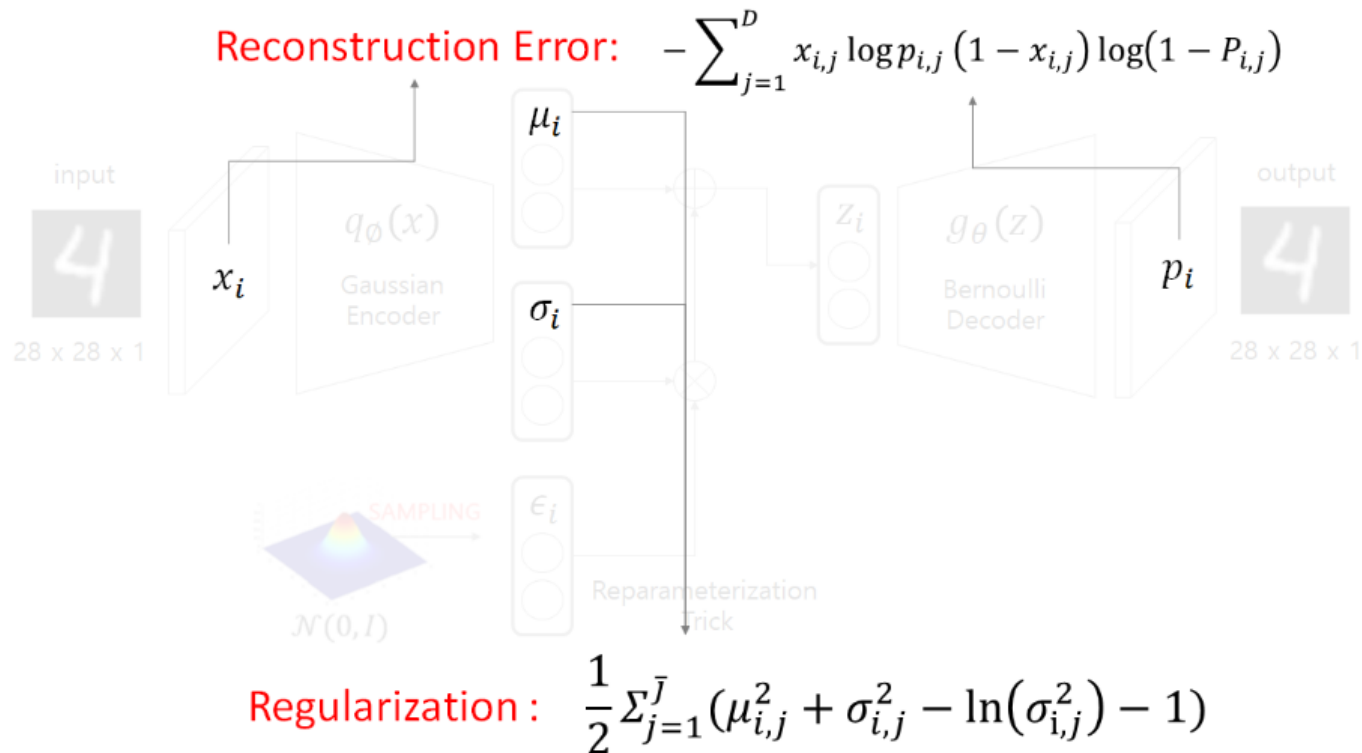
VAE의 실제 구조



VAE의 Loss Function



$$\arg \min_{\theta, \phi} \sum_i \underbrace{-\mathbb{E}_{q_{\phi}(z|x_i)}[\log(p(x_i|g_{\theta}(z)))]}_{\text{Reconstruction Error}} + \underbrace{KL(q_{\phi}(z|x_i)||p(z))}_{\text{Regularization}}$$



[참고] Kullback Leibler Divergence



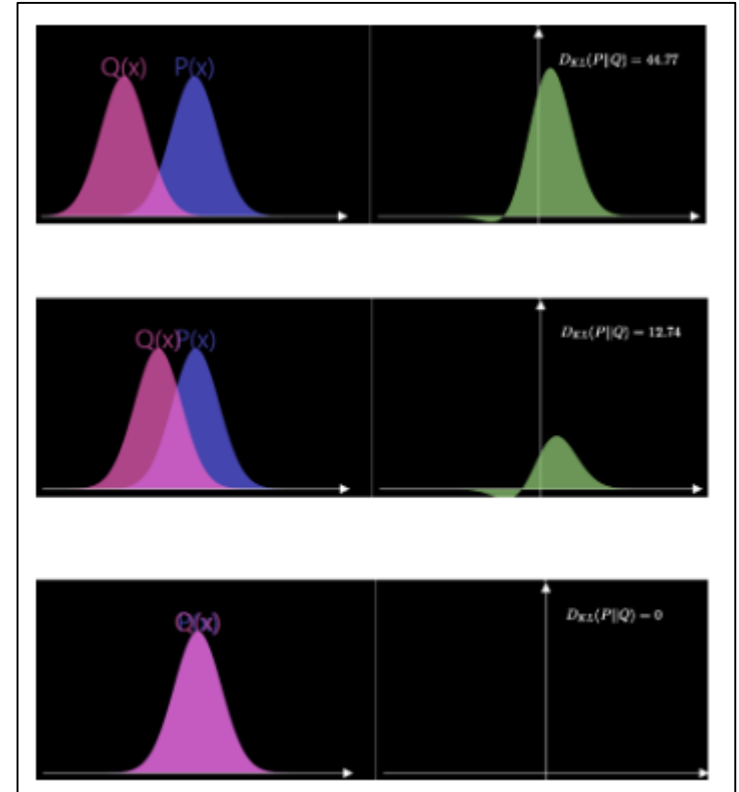
■ KL Divergence

- 두 확률 분포 간의 차이를 나타냄

$$\begin{aligned} D_{KL}(P\|Q) &= \sum_{x \in \chi} P(x) \log_b \left(\frac{P(x)}{Q(x)} \right) \\ &\Rightarrow - \sum_{x \in \chi} P(x) \log_b \left(\frac{Q(x)}{P(x)} \right) \\ &= - \sum_{x \in \chi} P(x) \log_b Q(x) + \sum_{x \in \chi} P(x) \log_b P(x) \end{aligned}$$

$$\Rightarrow \boxed{-E_P[\log_b Q(x)]} + \boxed{E_P[\log_b P(x)]} \longrightarrow \text{단순 p에 대한 정보 엔트로피}$$

Cross Entropy Loss (P에서 샘플링한 x를 통해 Q의 평균 정보량을 나타냄)



- Sampling Layer 생성

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

class Sampling(layers.Layer):

    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.random.normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon
```

■ Sampling Layer 생성

- Encoder

```
latent_dim = 2

encoder_inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(encoder_inputs)
x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Flatten()(x)
x = layers.Dense(16, activation="relu")(x)
z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var])
encoder = keras.Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")
encoder.summary()
```

- Decoder **Conv2DTranspose : 전치된 컨볼루션 레이어 → Deconvolution이라고도 함**

```
latent_inputs = keras.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 3, activation="relu", strides=2, padding="same")(x)
x = layers.Conv2DTranspose(32, 3, activation="relu", strides=2, padding="same")(x)
decoder_outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)
decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")
decoder.summary()
```

Variational Auto-Encoder 실습



■ VAE 모델 생성

- Class 활용 VAE 모델 및 Loss 함수 정의

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = keras.metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker,
        ]

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    keras.losses.binary_crossentropy(data, reconstruction), axis=(1, 2)
                )
            )
            kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
            total_loss = reconstruction_loss + kl_loss
            grads = tape.gradient(total_loss, self.trainable_weights)
            self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
            self.total_loss_tracker.update_state(total_loss)
            self.reconstruction_loss_tracker.update_state(reconstruction_loss)
            self.kl_loss_tracker.update_state(kl_loss)
        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
        }
```

Variational Auto-Encoder 실습



- MNIST 데이터 활용하여 학습

```
(x_train, _), (x_test, _) = keras.datasets.mnist.load_data()
mnist_digits = np.concatenate([x_train, x_test], axis=0)
mnist_digits = np.expand_dims(mnist_digits, -1).astype("float32") / 255

vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())
vae.fit(mnist_digits, epochs=30, batch_size=128)
```

```
Epoch 1/30
547/547 [=====] - 2s 3ms/step - loss: 267.1123 - reconstruction_loss: 210.6297 - kl_loss: 2.6868
Epoch 2/30
547/547 [=====] - 2s 3ms/step - loss: 175.8373 - reconstruction_loss: 167.6613 - kl_loss: 5.7406
Epoch 3/30
547/547 [=====] - 2s 3ms/step - loss: 168.5320 - reconstruction_loss: 161.7087 - kl_loss: 6.0236
Epoch 4/30
547/547 [=====] - 2s 3ms/step - loss: 165.1030 - reconstruction_loss: 158.4804 - kl_loss: 6.2564
Epoch 5/30
547/547 [=====] - 2s 3ms/step - loss: 163.0311 - reconstruction_loss: 156.3551 - kl_loss: 6.3594
Epoch 6/30
547/547 [=====] - 2s 3ms/step - loss: 161.3094 - reconstruction_loss: 154.8412 - kl_loss: 6.4112
Epoch 7/30
547/547 [=====] - 2s 3ms/step - loss: 160.1096 - reconstruction_loss: 153.5677 - kl_loss: 6.4803
Epoch 8/30
547/547 [=====] - 2s 3ms/step - loss: 158.9995 - reconstruction_loss: 152.5498 - kl_loss: 6.5168
Epoch 9/30
547/547 [=====] - 2s 3ms/step - loss: 158.0194 - reconstruction_loss: 151.7895 - kl_loss: 6.5271
```

Variational Auto-Encoder 실습



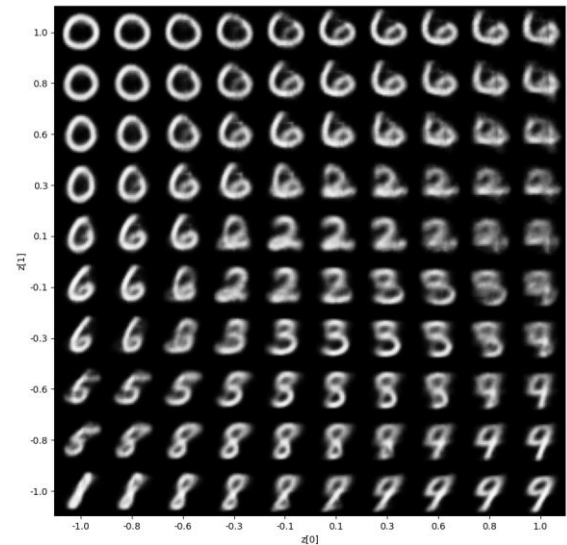
■ 학습 결과 확인

```
def plot_latent_space(vae, n=10, figsize=15):
    # display an n*n 2D manifold of digits
    digit_size = 28
    scale = 1.0
    figure = np.zeros((digit_size * n, digit_size * n))
    # linearly spaced coordinates corresponding to the 2D plot
    # of digit classes in the latent space
    grid_x = np.linspace(-scale, scale, n)
    grid_y = np.linspace(-scale, scale, n)[::-1]

    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):
            z_sample = np.array([[xi, yi]])
            x_decoded = vae.decoder.predict(z_sample)
            digit = x_decoded[0].reshape(digit_size, digit_size)
            figure[
                i * digit_size : (i + 1) * digit_size,
                j * digit_size : (j + 1) * digit_size,
            ] = digit

    plt.figure(figsize=(figsize, figsize))
    start_range = digit_size // 2
    end_range = n * digit_size + start_range
    pixel_range = np.arange(start_range, end_range, digit_size)
    sample_range_x = np.round(grid_x, 1)
    sample_range_y = np.round(grid_y, 1)
    plt.xticks(pixel_range, sample_range_x)
    plt.yticks(pixel_range, sample_range_y)
    plt.xlabel("z[0]")
    plt.ylabel("z[1]")
    plt.imshow(figure, cmap="Greys_r")
    plt.show()

plot_latent_space(vae)
```



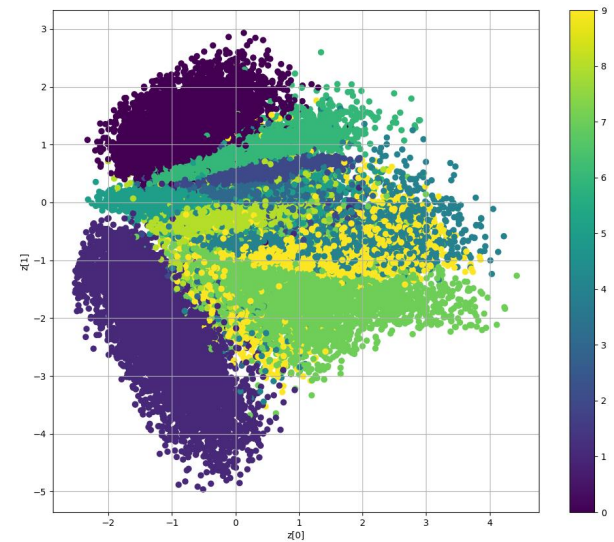
잠재벡터별 생성값

■ 학습 결과 확인

```
def plot_label_clusters(vae, data, labels):  
    # display a 2D plot of the digit classes in the latent space  
    z_mean, _, _ = vae.encoder.predict(data)  
    plt.figure(figsize=(12, 10))  
    plt.scatter(z_mean[:, 0], z_mean[:, 1], c=labels)  
    plt.colorbar()  
    plt.xlabel("z[0]")  
    plt.ylabel("z[1]")  
    plt.show()  
  
(x_train, y_train), _ = keras.datasets.mnist.load_data()  
x_train = np.expand_dims(x_train, -1).astype("float32") / 255  
  
plot_label_clusters(vae, x_train, y_train)
```



잠재벡터가 2차원인 경우



숫자별 잠재벡터 군집

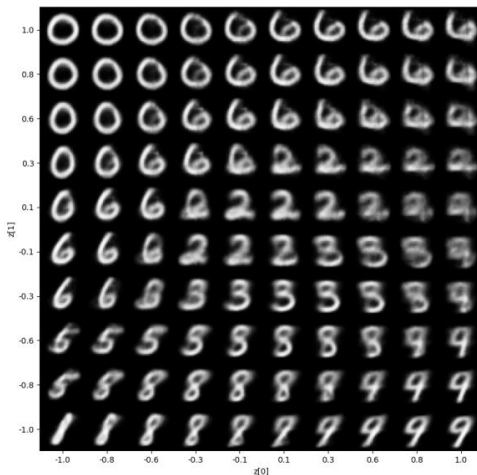
Chap.8 실습 (3)



각 실습문제에 대해 *code(+ 주석), 그래프, 분석 내용* 등은 필수적으로 포함시킬 것

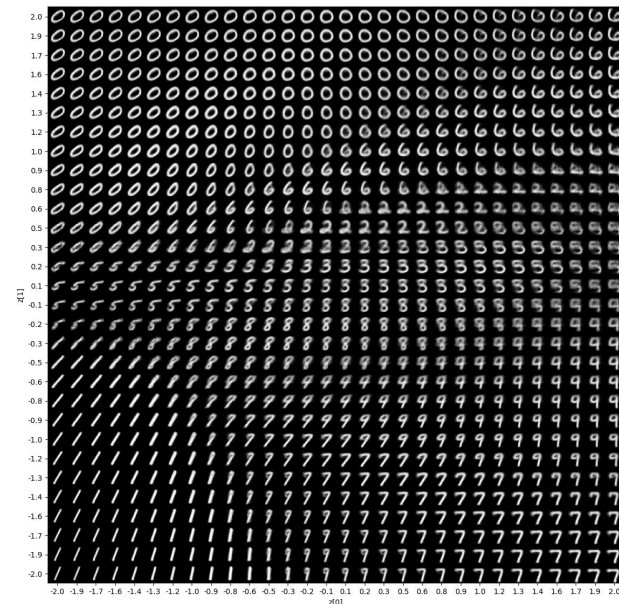
실습 #4) VAE를 활용한 MNIST 데이터 생성

- 앞선 실습을 따라하여 각 Latent vector에 따른 숫자 데이터 생성
- 코드 내용이 복잡하고 어렵기 때문에, 파이썬에 대한 추가적인 공부 요망
ex) Class, 데코레이터 등등
- 잠재벡터를 -2~2 사이에서 총 900개의 숫자 데이터 생성
- VAE 구조 및 파라미터 조정에 따른 결과 비교 분석
- 각 잠재벡터 마다 알맞은 숫자가 생성되었는지 확인 → 다음 페이지 참고
- **단순 과제 제출 목적이 아닌, VAE에 대한 이해를 목적으로 진행 요망**



잠재벡터 -1~1
총 100개 생성

잠재벡터 -2~2
총 900개 생성



[참고] Latent vector별 생성 숫자

